

Project - High Level Design

On

Deploy Manufacturing App to Kubernetes

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	AADITI BHALE	EN22CS303001
2.	ADITYA PUROHIT	EN22CS303007
3.	ABHIGYAN SINGH THAKUR	EN22CS303004
4.	DHRUV JAIN	EN22CS304022
5.	DIVYANSH TYAGI	EN22CS304024

Group Name: 06D11

Project Number:032

Industry Mentor Name: AASHRUTI SHAH

University Mentor Name: SHYAM PATEL

Academic Year:2022-2026

Table of Contents

1. Introduction.

- 1.1. Scope of the document.
- 1.2. Intended Audience
- 1.3. System overview.

2. System Design.

- 2.1. Application Design
- 2.2. Process Flow.
- 2.3. Information Flow.
- 2.4. Components Design
- 2.5. Key Design Considerations
- 2.6. API Catalogue.

3. Data Design.

- 3.1. Data Model
- 3.2. Data Access Mechanism
- 3.3. Data Retention Policies
- 3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

- 7.1. Security Aspects
- 7.2. Performance Aspects

8. References

1.Introduction:

This document describes the Low-Level Design (LLD) of the Manufacturing Application deployed using Kubernetes. It explains internal components, workflows, API definitions, data handling, system interactions, and deployment mechanisms in detail.

1.1 Scope of the Document

This document covers:

- Detailed application design
- Containerization approach
- Kubernetes object configuration
- Data flow and process flow
- API endpoints
- Deployment strategy
- Security and performance considerations

It does not cover CI/CD automation implementation in depth.

1.2 Intended Audience

This document is intended for:

- DevOps Engineers
- Developers
- Kubernetes Administrators
- Technical Review Committee
- Project Evaluators

1.3 System Overview

The system is a containerized Flask-based Manufacturing Application deployed on Kubernetes. It supports:

- High availability
- Rolling updates
- Health monitoring
- External access via NodePort

Architecture follows a container orchestration model.

2. System Design

2.1 Application Design

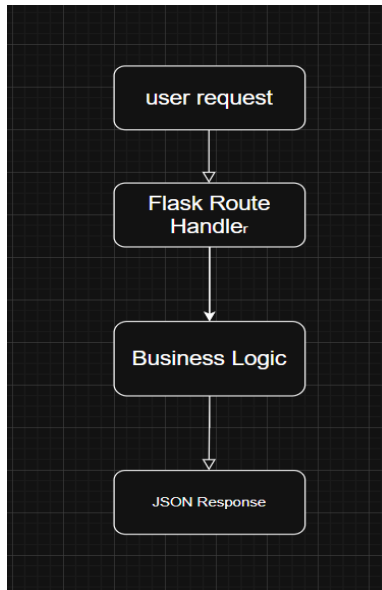
The application is built using Python Flask.

Core Endpoints:

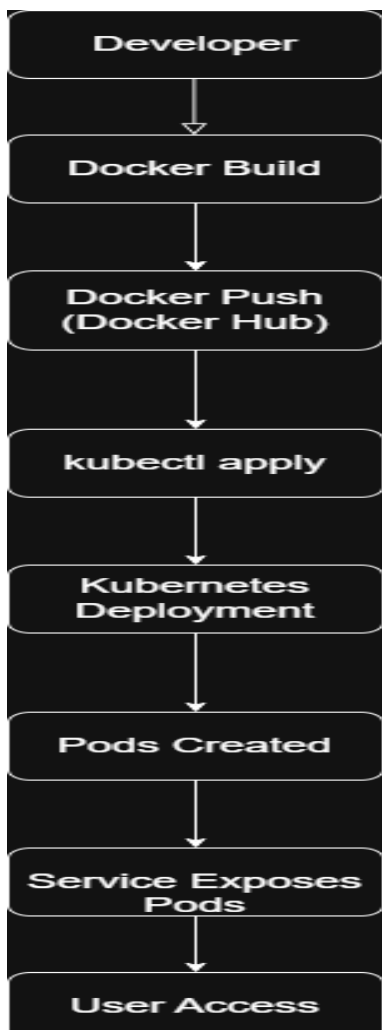
Endpoint	Method	Purpose
/	GET	Returns application version
/status	GET	Returns machine status
/health	GET	Used for liveness/readiness

Application runs on port **5000**.

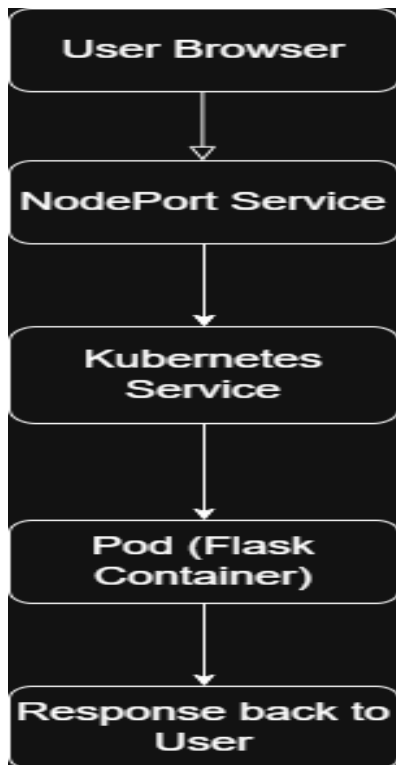
Application Flow Diagram



2.2 Process Flow - Deployment Flow



2.3 Information Flow



2.4 Component Design

Component 1: Flask Application

- Handles HTTP requests
- Stateless
- Health endpoint enabled

Component 2: Docker

- Base Image: python:3.10-slim
- Exposes port 5000
- Runs app.py

Component 3: Docker Hub

- Stores container images
- Versioned tagging (v1, v2)

Component 4: Kubernetes Deployment

- Manages replicas
- Rolling update strategy
- Liveness & readiness probes

Component 5: Kubernetes Service

- Type: NodePort
- Exposes application externally

2.5 Key Design Considerations

- Stateless design for scalability
- Zero-downtime rolling updates
- Container image versioning
- Health checks for reliability
- Minimal resource usage

2.6 API Catalogue

1. Home API

```
GET /  
Response:  
"Manufacturing App running - Version v1"
```

2. Status API

```
GET /status  
Response:  
{  
  "machine": "operational",  
  "temperature": "normal",  
  "version": "v1"  
}
```

3. Health API

```
GET /health  
Response:  
"OK"
```

Used by:

- Liveness probe
- Readiness probe

3. Data Design

3.1 Data Model

This system is currently stateless.

No database is used.

Data returned is static JSON.

Future data model (if DB added):

```
Machine  
-----  
machine_id  
status  
temperature  
timestamp
```

3.2 Data Access Mechanism

- Direct in-memory JSON responses
- No persistent storage

3.3 Data Retention Policies

- No data storage
- No logs persisted beyond pod lifecycle

3.4 Data Migration

Not applicable (stateless system)

4. Interfaces

The Manufacturing Application interacts with external users and internal system components through well-defined interfaces. These interfaces ensure modularity, maintainability, and scalability.

The interfaces are categorized into:

1. External Interfaces
2. Internal Interfaces
3. Infrastructure Interfaces
4. Deployment Interfaces

4.1 External API Interface

4.1.1 Home Endpoint Interface



Figure 4.1: Home API Endpoint Output (Version Verification Interface)

The root endpoint (/) serves as the primary interface between the end user and the application. When accessed via a web browser, it returns the current application version running inside the container. This confirms successful application deployment and accessibility.

The request is handled by the Flask application running inside the Kubernetes pod (or Docker container during local testing).

4.1.2 Status API Interface

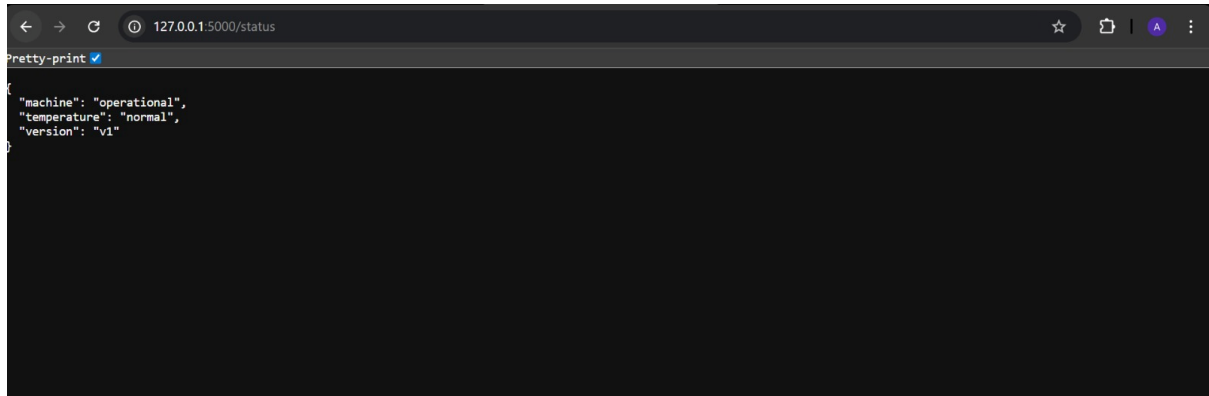


Figure 4.2: Status API JSON Response Interface

The /status endpoint provides operational details of the manufacturing system in JSON format. This interface allows external systems or monitoring tools to retrieve system health and operational status programmatically.

The response includes:

- Machine operational state
- Temperature status
- Application version

This interface demonstrates RESTful API communication.

4.1.3 Health Check Interface



Figure 4.3: Health Check Interface Used by Kubernetes Probes

The /health endpoint acts as an internal monitoring interface. Kubernetes uses this endpoint for liveness and readiness probes to ensure that traffic is routed only to healthy pods.

This interface is critical for zero-downtime deployments and self-healing mechanisms.

4.2 Container Registry Interface

4.2.1 Docker Hub Repository Interface

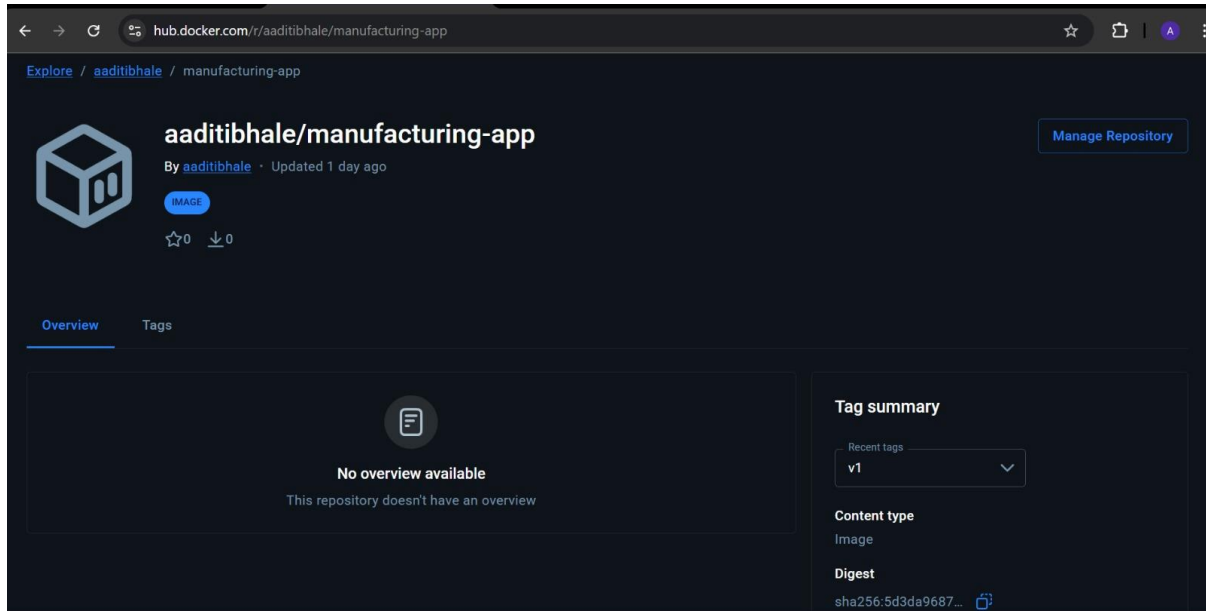


Figure 4.4: Docker Hub Repository Interface Showing Image Tag v1

Docker Hub acts as the container registry interface between the development environment and Kubernetes cluster. The application image is pushed to Docker Hub, where it is versioned and stored.

Kubernetes pulls the container image from Docker Hub during deployment.

This interface ensures portability and centralized image management.

4.3 Deployment Interface

4.3.1 Docker CLI Push Interface

```
[node1] (local) root@192.168.0.14 ~/06D11/Group 06 D11
$ docker push aaditibhale/manufacturing-app v1
'docker push' requires exactly 1 argument.
See 'docker push --help'.

Usage:  docker push [OPTIONS] NAME[:TAG]

Upload an image to a registry
[node1] (local) root@192.168.0.14 ~/06D11/Group 06 D11
```

Figure 4.5: Docker Push Command Interface Demonstrating Tag Syntax

4.4 Kubernetes Deployment Interface

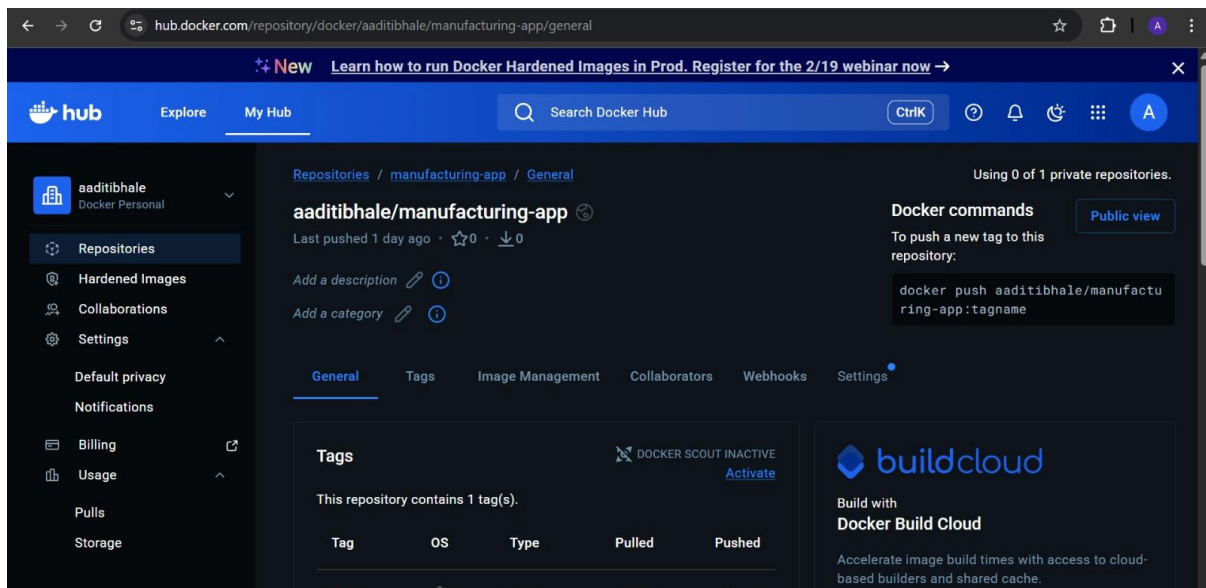


Figure 4.6: Container Registry Interface Used by Kubernetes Deployment

Kubernetes interacts with Docker Hub through image pull operations. When the deployment.yaml file is applied using kubectl, Kubernetes automatically pulls the specified image version from Docker Hub.

This interface ensures that the correct container image version is deployed in the cluster.

5.State and Session Management

5.1 Overview

The Manufacturing Application is designed as a stateless REST-based service. This means that each client request is independent and does not rely on previous interactions.

The application does not store:

- User sessions
- Authentication tokens (currently)
- Persistent state
- Application memory between requests

This design aligns with Kubernetes scalability and cloud-native architecture principles.

5.2 Why Stateless Design Was Chosen

The stateless design provides:

- Horizontal scalability
- Easy rolling updates
- No sticky sessions required
- Load-balanced traffic distribution
- Better fault tolerance

In Kubernetes, pods may terminate or restart at any time. A stateless architecture ensures no data loss during pod restarts.

5.3 Session Handling Strategy (Future Enhancement)

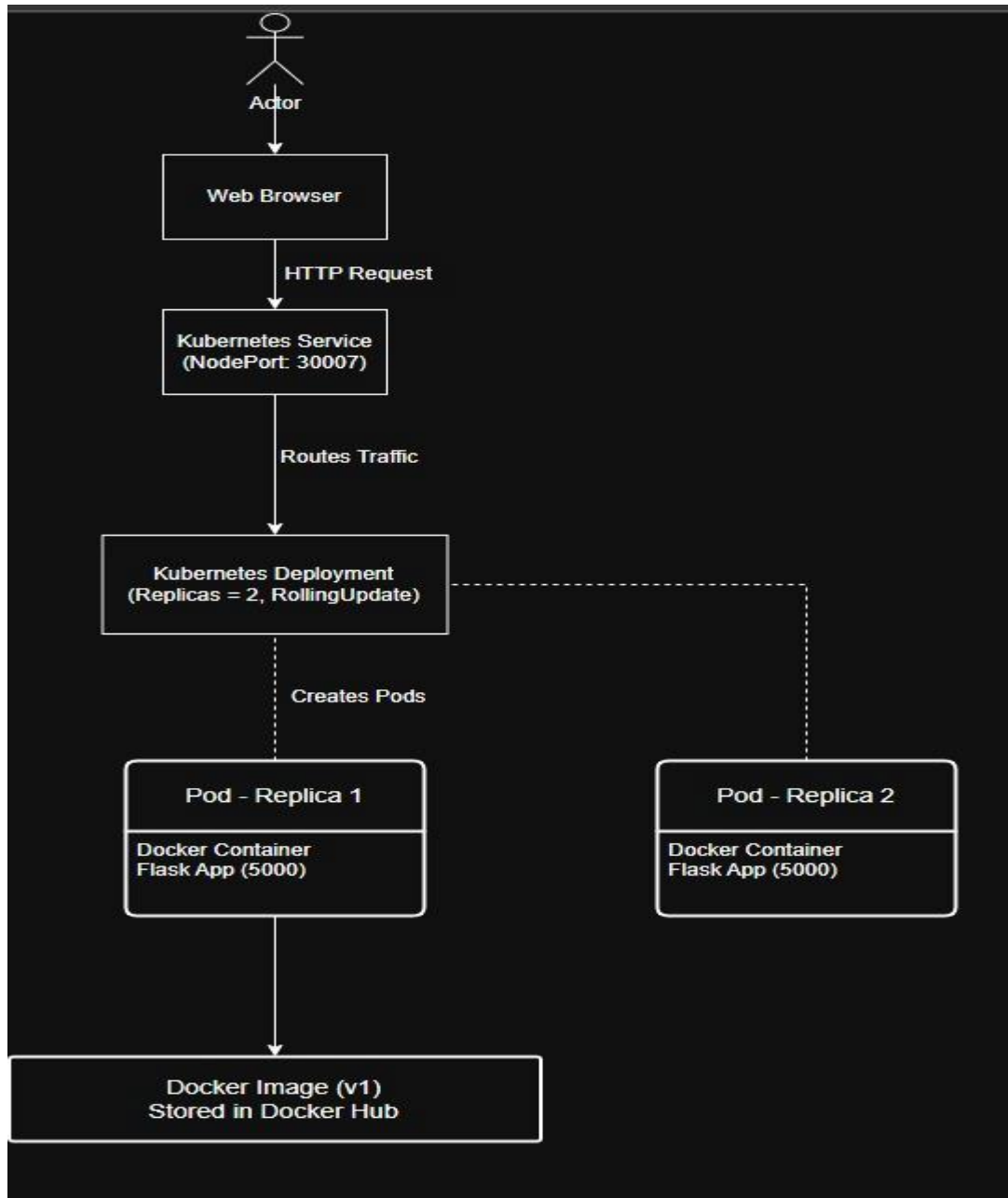
If user authentication is implemented, the following approach can be used:

- JWT-based token authentication
- Session stored client-side
- Redis-based distributed session store

Currently, no session management mechanism is implemented.

5.4 Architecture Design

Each HTTP request follows this flow:



No session information is stored on the server.

5.5 Impact on Kubernetes Deployment

Since the application is stateless:

- Replicas can scale without synchronization
- Rolling updates do not disrupt users
- No shared memory between pods is required

6. Caching

The current version of the Manufacturing Application does not implement caching. Since the application returns static JSON responses and does not interact with a database or perform computationally intensive operations, caching is not required at this stage. Each incoming request is processed independently by the Flask application running inside the container, and the response is generated in real time.

However, the system design allows for future integration of caching mechanisms if required. In a production-scale environment, caching can be introduced at the application level using in-memory caching tools such as Redis, or at the infrastructure level using reverse proxies or Kubernetes Ingress controllers. This would help reduce response latency, decrease container CPU utilization, and improve overall performance under high traffic conditions.

The stateless nature of the application makes it compatible with distributed caching systems if implemented in the future.

6.1 Current Implementation

The current version of the Manufacturing Application does not implement caching.

Each request is processed independently and returns a real-time response.

6.2 Why Caching Is Not Implemented

The application:

- Returns static JSON responses
- Does not query a database
- Does not perform expensive computations

Therefore, caching is not necessary in the current implementation.

6.3 Future Caching Strategy

For production-grade systems, caching can be implemented at multiple levels:

6.3.1 Application-Level Caching

- Redis integration
- In-memory caching using Flask-Caching

6.3.2 Reverse Proxy Caching

- NGINX caching
- Ingress controller caching

6.3.3 API Response Caching

- Cache frequent /status calls
- Reduce container CPU usage

6.4 Benefits of Caching

- Reduced latency
- Improved performance
- Lower CPU utilization
- Reduced load on backend services

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system beyond functional behavior. For this Manufacturing Application, the primary non-functional considerations include scalability, reliability, availability, performance, and security.

The application is designed to be stateless, enabling horizontal scalability through Kubernetes replicas. Multiple pod instances ensure high availability and fault tolerance. Kubernetes automatically recreates failed pods, ensuring service continuity.

Zero-downtime deployment is achieved through a rolling update strategy. During version updates, new pods are created before terminating old ones, ensuring uninterrupted service for end users.

From a security perspective, the application runs inside isolated Docker containers and does not store sensitive information. The container image is version-controlled and pulled securely from Docker Hub. Kubernetes service abstraction prevents direct exposure of pods, enhancing network security.

Performance is optimized by using a lightweight Flask framework and a minimal Docker image, ensuring fast startup times and low resource consumption. The system is designed to be extensible, allowing integration with monitoring tools, secure ingress controllers, and advanced authentication mechanisms in future enhancements.

7.1 Security Aspects

7.1.1 Container Security

- Uses official Python base image
- Minimal dependencies installed
- No hardcoded credentials
- No sensitive data stored in image

7.1.2 Kubernetes Security

- Pods are isolated
- Service abstraction prevents direct pod exposure
- Only NodePort is exposed
- Internal communication within cluster network

7.1.3 Network Security

- Traffic limited to defined NodePort
- No external database connections
- Health endpoint accessible only through cluster routing

Future Enhancements:

- HTTPS using Ingress
- TLS encryption
- Network policies
- Role-Based Access Control (RBAC)

7.1.4 Image Versioning Security

- Version-based tagging (v1, v2)
- Prevents accidental overwrite
- Supports controlled rollout

7.2 Performance Aspects

7.2.1 Scalability

- Multiple pod replicas
- Stateless design
- Kubernetes load balancing

7.2.2 High Availability

- Minimum 2 replicas
- Self-healing pods
- Rolling update strategy

7.2.3 Zero-Downtime Deployment

Implemented using:

- RollingUpdate strategy
- maxUnavailable = 0
- Readiness probe
- Liveness probe

This ensures:

- New pod becomes ready before old pod termination
- Continuous service availability

7.2.4 Resource Efficiency

- Lightweight Flask framework
- Minimal CPU consumption
- Slim Docker image
- Fast container startup time

7.2.5 Reliability

- Kubernetes auto-restarts failed pods
- Health check monitoring
- Controlled image deployment

7.2.6 Observability (Future Enhancement)

Future enhancements may include:

- Prometheus monitoring
- Grafana dashboards
- Centralized logging (ELK stack)

8. References

- [1] Kubernetes, “Kubernetes Documentation,” Kubernetes Official Website. [Online]. Available: <https://kubernetes.io/docs/>. Accessed: Feb. 19, 2026.
- [2] Docker Inc., “Docker Documentation,” Docker Official Website. [Online]. Available: <https://docs.docker.com/>. Accessed: Feb. 19, 2026.
- [3] Docker Inc., “Docker Hub Documentation,” Docker Hub. [Online]. Available: <https://docs.docker.com/docker-hub/>. Accessed: Feb. 19, 2026.
- [4] Pallets Projects, “Flask Documentation,” Flask Official Documentation. [Online]. Available: <https://flask.palletsprojects.com/>. Accessed: Feb. 19, 2026.
- [5] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [6] C. Richardson, *Microservices Patterns: With examples in Java*, Manning Publications, 2018.
- [7] The Linux Foundation, “Cloud Native Computing Foundation (CNCF) Resources,” CNCF. [Online]. Available: <https://www.cncf.io/>. Accessed: Feb. 19, 2026.